

prepare_qat_fx#

[https://pytorch.org/docs/stable/generated/torch.ao.quantization.quantize_fx.](https://pytorch.org/docs/stable/generated/torch.ao.quantization.quantize_fx.html)

Description:

Prepare a model for quantization aware training model (*) – torch.nn.Module model qconfig_mapping (*) – see prepare_fx() example_inputs (*) – see prepare_fx() prepare_custom_config (*) – see prepare_fx()

Function Signature

```
class
torch.ao.quantization.quantize_fx.prepare_qat_fx(model,
qconfig_mapping, example_inputs, prepare_custom_config=None,
backend_config=None)[source]#
```

Parameters

Returns

Return type

Returns:

GraphModule

Code Examples

```
import torch
from torch.ao.quantization import
get_default_qat_qconfig_mapping
from torch.ao.quantization.quantize_fx import prepare_qat_fx

class Submodule(torch.nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.linear = torch.nn.Linear(5, 5)

    def forward(self, x):
        x = self.linear(x)
        return x

class M(torch.nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.linear = torch.nn.Linear(5, 5)
        self.sub = Submodule()

    def forward(self, x):
        x = self.linear(x)
        x = self.sub(x) + x
        return x
```

```

# initialize a floating point model
float_model = M().train()
# (optional, but preferred) load the weights from pretrained
model
# float_model.load_weights(...)

# define the training loop for quantization aware training
def train_loop(model, train_data):
    model.train()
    for image, target in data_loader:
        ...

# qconfig is the configuration for how we insert observers for a
particular
# operator
# qconfig = get_default_qconfig("fbgemm")
# Example of customizing qconfig:
# qconfig = torch.ao.quantization.QConfig(
#
# activation=FakeQuantize.with_args(observer=MinMaxObserver.with_a
# args(dtype=torch.qint8)),
#
# weight=FakeQuantize.with_args(observer=MinMaxObserver.with_args(
# dtype=torch.qint8)))
# `activation` and `weight` are constructors of observer module

# qconfig_mapping is a collection of quantization
configurations, user can
# set the qconfig for each operator (torch op calls, functional
calls, module calls)
# in the model through qconfig_mapping
# the following call will get the qconfig_mapping that works
best for models
# that target "fbgemm" backend
qconfig_mapping = get_default_qat_qconfig_mapping("fbgemm")

```

```
# We can customize qconfig_mapping in different ways, please
take a look at
# the docstring for :func:`~torch.ao.quantization.prepare_fx`
for different ways
# to configure this

# example_inputs is a tuple of inputs, that is used to infer the
type of the
# outputs in the model
# currently it's not used, but please make sure
model(*example_inputs) runs
example_inputs = (torch.randn(1, 3, 224, 224),)

# TODO: add backend_config after we split the backend_config for
fbgemm and qnnpack
# e.g. backend_config = get_default_backend_config("fbgemm")
# `prepare_qat_fx` inserts observers in the model based on
qconfig_mapping and
# backend_config, if the configuration for an operator in
qconfig_mapping
# is supported in the backend_config (meaning it's supported by
the target
# hardware), we'll insert fake_quantize modules according to the
qconfig_mapping
# otherwise the configuration in qconfig_mapping will be ignored
# see :func:`~torch.ao.quantization.prepare_fx` for a detailed
explanation of
# how qconfig_mapping interacts with backend_config
prepared_model = prepare_qat_fx(float_model, qconfig_mapping,
example_inputs)
# Run training
train_loop(prepared_model, train_loop)
```

Detailed Documentation

Documentation

Prepare a model for quantization aware training

model (*) – torch.nn.Module model

qconfig_mapping (*) – see prepare_fx()

example_inputs (*) – see prepare_fx()

prepare_custom_config (*) – see prepare_fx()

backend_config (*) – see prepare_fx()

A GraphModule with fake quant modules (configured by qconfig_mapping and backend_config), ready for quantization aware training

Prepare a model for quantization aware training

model (*) – torch.nn.Module model

model (*) – torch.nn.Module model

qconfig_mapping (*) – see prepare_fx()

qconfig_mapping (*) – see prepare_fx()

example_inputs (*) – see prepare_fx()

example_inputs (*) – see prepare_fx()

prepare_custom_config (*) – see prepare_fx()

prepare_custom_config (*) – see prepare_fx()

backend_config (*) – see prepare_fx()

backend_config (*) – see prepare_fx()

A GraphModule with fake quant modules (configured by qconfig_mapping and backend_config), ready for quantization aware training